CASE STUDY-BASED ANALYSIS AND DESIGN USING PUSHDOWN AUTOMATA AND CONTEXT-FREE GRAMMAR

CASE STUDY REPORT

Submitted by

Swathi Palanisamy (RA2411026050013)

FORMAL LANGUAGE AND AUTOMATA

In Partial Fulfilment of the Requirements
for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING WITH SPECIALIZATION IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

TIRUCHIRAPPALLI



SCHOOL OF COMPUTING
FACULTY OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
TIRUCHIRAPPALLI, 621105

October 2026

ABSTRACT

This case study report presents three applications of Context-Free Grammar (CFG) and Pushdown Automata (PDA): arithmetic expression validation, balanced-parentheses checking, and palindrome recognition over the alphabet {a, b}. Each application is specified as a formal language, modelled using grammar productions and stack-based processing, and validated through accepted and rejected test strings. The submitted Python implementation produces 8/8 correct test outcomes for arithmetic expressions, 8/8 for balanced parentheses, and 10/10 for palindrome strings. The report also explains the role of the stack, the distinction between deterministic and nondeterministic PDA processing, and why finite automata alone are insufficient for the unbounded matching requirements in these examples.

TABLE OF CONTENTS

1. Introduction

2. Objectives and Scope

3. Theoretical Background

4. Case Study 1 – Arithmetic Expression Validation

5. Case Study 2 – Balanced Parentheses Checker

6. Case Study 3 – Palindrome Checker

7. Comparative Analysis

8. Results and Discussion

9. Conclusion

10. References

1. INTRODUCTION

Formal Language and Automata theory provides mathematical models for describing and recognizing structured patterns in strings. Regular languages can be recognized by finite automata, while context-free languages require the expressive power of a stack-based model such as a Pushdown Automaton.

This task studies three practical validation problems. Arithmetic expressions require correct ordering of operands, operators, and nested parentheses. Balanced-parentheses validation requires remembering unmatched opening symbols. Palindrome recognition requires comparing the first half of a string with its second half in reverse order. These tasks demonstrate how CFGs describe valid strings and how PDAs process them.

2. OBJECTIVES AND SCOPE

- Define the target language for each selected application.
- Construct suitable CFG productions and explain their meaning.
- Describe PDA states, stack symbols, and stack operations.
- Show successful processing examples and rejected inputs.
- Discuss deterministic and nondeterministic processing and relevant limitations.
- Validate each implementation using the test cases included in the submitted Python notebook.

3. THEORETICAL BACKGROUND

3.1 Context-Free Grammar

A Context-Free Grammar is a four-tuple G = (V, Σ, P, S), where V is the set of non-terminals, Σ is the set of terminals, P is the set of productions, and S is the start symbol. A production replaces one non-terminal with a sequence of terminals and/or non-terminals.

3.2 Pushdown Automaton

A Pushdown Automaton is a finite-state machine equipped with a stack. A PDA can push symbols onto the stack, inspect or pop the stack top, and change states while reading input. The stack provides memory for nested structures and matching relationships that cannot be handled by a finite amount of state alone.

3.3 Why a Finite Automaton Is Not Sufficient

A finite automaton has only a fixed number of states and no unbounded auxiliary memory. It cannot generally keep track of an arbitrary number of opening parentheses until matching closing parentheses arrive. Similarly, it cannot compare an arbitrarily long first half of a palindrome with the reverse of its second half. A PDA uses its stack for these matching operations. The arithmetic grammar also includes recursively nested parentheses, which are naturally represented by a CFG and PDA.

The implementation in the notebook uses a predictive stack simulation for arithmetic expressions, a deterministic push/pop procedure for parentheses, and a nondeterministic PDA search for palindromes.

4. CASE STUDY 1 – ARITHMETIC EXPRESSION VALIDATION

4.1 Problem Definition

The validation process is important because an expression can contain familiar mathematical symbols and still be syntactically incorrect. For example, an operator must occur between two valid operands, and an opening parenthesis must be matched by a closing parenthesis. The parser therefore checks the structure of the expression rather than calculating its numerical value. This distinction is useful in calculators, interpreters, compilers, and educational tools that must detect malformed expressions before further processing.

The objective is to determine whether an input string follows the syntax of a basic arithmetic expression. The accepted expressions may contain identifiers or numeric literals, the operators +, -, *, and /, and parentheses for grouping. Whitespace is ignored during tokenization.

4.2 Target Language

The language consists of syntactically valid arithmetic expressions formed from operands, binary operators, and properly nested parentheses. The implementation tokenizes numeric values and identifiers as the terminal id.

4.3 Context-Free Grammar

The grammar is designed to remove ambiguity in the usual precedence rules by separating the expression into different levels. At the highest level, E handles addition and subtraction. T handles multiplication and division, while F handles operands and grouped expressions. Because E contains T rather than directly containing F, multiplication and division are grouped more tightly than addition and subtraction. The recursive use of E inside parentheses allows expressions of arbitrary nesting depth to be represented by the grammar.

$$
\begin{aligned}
E &\rightarrow T\ E' \\
E' &\rightarrow +\ T\ E' \mid -\ T\ E' \mid \varepsilon \\
T &\rightarrow F\ T' \\
T' &\rightarrow *\ F\ T' \mid /\ F\ T' \mid \varepsilon \\
F &\rightarrow (\ E\ ) \mid \mathrm{id}
\end{aligned}
$$

E represents an expression, T represents a term, and F represents a factor. E′ allows repeated addition or subtraction. T′ allows repeated multiplication or division. F is either a parenthesized expression or an operand. This structure gives multiplication and division higher precedence than addition and subtraction.

4.4 PDA Design and Stack Behaviour

During parsing, the stack acts as a record of the grammar symbols that are still expected. When a non-terminal is on top of the stack, the parser selects a production using the next input token. The right-hand side of that production is placed on the stack in reverse order so that its leftmost symbol is processed next. When a terminal is on top, it must be identical to the current input token; otherwise the parser rejects the expression. This process continues until the input and stack end together. A mismatch, an invalid token, or leftover stack symbols causes rejection.

The predictive PDA begins with the stack containing the end marker $ and start symbol E. It reads the next token and expands a non-terminal according to the grammar and lookahead token. When the top of the stack

is a terminal, it must match the current input token. The input is accepted only when the end marker is matched, the stack is empty, and the complete input has been consumed.

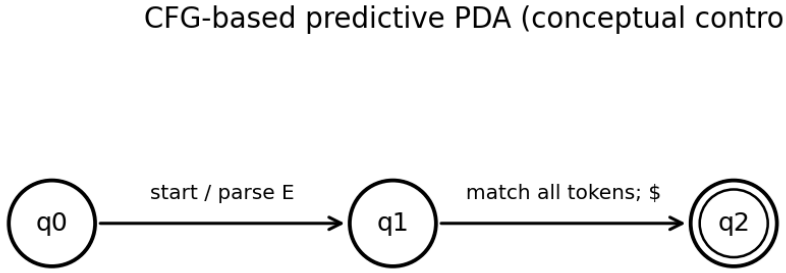


Figure 1. Conceptual control flow of the CFG-based predictive PDA.

Component	Description
Initial stack	$, E
Non-terminals	E, E′, T, T′, F
Terminals	+, -, *, /, (,), id, $
Acceptance	All input consumed and stack empty after matching $
Precedence	* and / have higher precedence than + and -

4.5 Worked Derivation

For the input (a+b)*c, the grammar can derive the expression as follows:

E ⇒ T E′
⇒ F T′ E′
⇒ (E) T′ E′
⇒ (T E′) T′ E′
⇒ (F T′ E′) T′ E′
⇒ (id T′ E′) T′ E′
⇒ (id + T E′) T′ E′
⇒ (id + id) * F T′ E′
⇒ (id + id) * id

The derivation shows that the subexpression a+b is grouped by parentheses and then multiplied by c. The parser's execution trace in the notebook consumes all tokens and reaches the final $ marker, so the string is accepted.

4.6 Test Case Validation

Input	Expected	Actual	Status
a+b	Accepted	Accepted	PASS
(a+b)*c	Accepted	Accepted	PASS
12+5*3	Accepted	Accepted	PASS
a*(b+c)	Accepted	Accepted	PASS

(a+b	Rejected	Rejected	PASS
a++b	Rejected	Rejected	PASS
*a+b	Rejected	Rejected	PASS
a+b)	Rejected	Rejected	PASS

Result: 8 test cases were executed. All 8 passed and no failures were reported.

4.7 Analysis

The grammar is suitable for the stated syntax because it represents nested grouping and establishes operator precedence. The grammar is also structured to support predictive parsing: the alternatives for E′, T′, and F can be selected using the next token in the supported expression format. However, acceptance only confirms grammatical form. It does not prove that an identifier has been declared, that a divisor is non-zero, or that an operation is meaningful for the operand types. Those checks belong to semantic analysis and evaluation, which are outside the scope of this case study.

- The grammar separates expressions, terms, and factors to encode operator precedence.
- The predictive stack mechanism checks the grammatical structure while consuming tokens.
- Unmatched parentheses, consecutive operators, and an operator in the first position are rejected.
- The implementation performs syntax validation; it does not evaluate the numerical result or check semantic conditions such as division by zero.

5. CASE STUDY 2 – BALANCED PARENTHESES CHECKER

5.1 Problem Definition

Balanced-parenthesis checking is a common operation in source-code editors, compilers, mathematical notation, and document-processing systems. A string is not balanced merely because it contains the same number of opening and closing symbols: the order must also be correct. For instance, the sequence ')(' has one opening and one closing parenthesis but is invalid because the closing symbol appears first. The checker must remember how many opening symbols are waiting to be matched and reject an unmatched closing symbol immediately.

The objective is to verify whether every opening parenthesis '(' in an input string has a corresponding closing parenthesis ')' in the correct order. A closing parenthesis cannot appear when there is no unmatched opening parenthesis.

5.2 Target Language

L = { w ∈ {(,)}* | w is properly balanced }. The empty string ε is also balanced.

5.3 Context-Free Grammar

This grammar describes both nesting and concatenation. The inner S in (S) generates the contents enclosed by a matching pair, while the final S permits another balanced sequence to follow it. The empty production is essential because it allows an empty region inside a pair and also permits the complete empty string. Repeated application of the recursive production can generate strings with any finite nesting depth, such as (), (()), and (()()).

S → (S)S | ε

The production (S)S creates one matched pair enclosing a balanced string, followed by another balanced string. The ε production allows the empty string.

5.4 DPDA Design

The stack provides exactly the memory needed for this task. Each opening parenthesis adds one pending obligation to the stack, and each closing parenthesis removes the most recent pending opening parenthesis. This last-in, first-out behaviour naturally enforces nesting: the most recently opened parenthesis must be closed first. The procedure is deterministic because the next action is determined by the current input symbol and whether the stack is empty. At end of input, a non-empty stack indicates that one or more opening parentheses were never closed.

The implementation uses a stack to store unmatched opening parentheses. On reading '(', it pushes '(' onto the stack. On reading ')', it pops one opening parenthesis. If the stack is empty when ')' is read, the input is rejected immediately. Any symbol other than '(' or ')' is also rejected. At the end, the input is accepted only if the stack is empty.

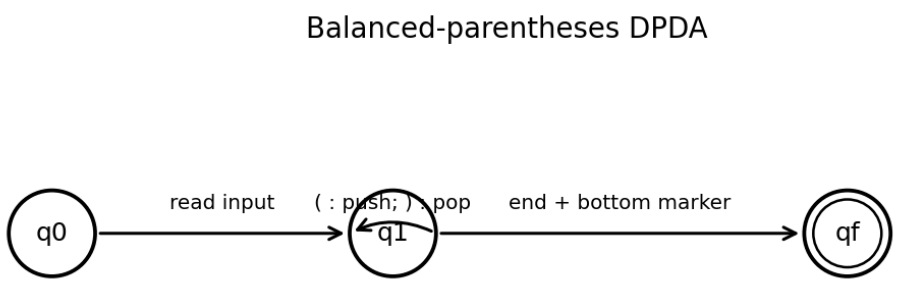


Figure 2. Simplified state flow for balanced-parentheses validation.

Element	Definition
Input alphabet	Σ = {(,)}
Stack symbol	((opening parenthesis)
Push operation	Read '(' and push '('
Pop operation	Read ')' and pop '(' if available
Reject condition	')' with empty stack, invalid symbol, or non-empty stack at end
Accept condition	Entire input consumed and stack empty

5.5 Worked Stack Trace

For the input ((())), the stack changes as each symbol is processed:

Step	Input symbol	Action	Stack after action
1	(	PUSH '('	[(]
2	(	PUSH '('	[(, (]
3	(	PUSH '('	[(, (, (]
4	)	POP '('	[(, (]
5	)	POP '('	[(]
6	)	POP '('	[]

After the final symbol, the stack is empty. Therefore, ((())) is accepted.

5.6 Test Case Validation

Input	Expected	Actual	Status
()	Accepted	Accepted	PASS
(())	Accepted	Accepted	PASS
()()	Accepted	Accepted	PASS
((()))	Accepted	Accepted	PASS
((	Rejected	Rejected	PASS
())	Rejected	Rejected	PASS
)(	Rejected	Rejected	PASS

(())	Rejected	Rejected	PASS

Result: 8 test cases were executed. All 8 passed and no failures were reported.

5.7 Analysis

The method detects both major error types: an early closing parenthesis and an opening parenthesis left unmatched at the end. It also rejects characters outside the specified alphabet, since the target language contains only '(' and ')'. A finite automaton cannot recognize this language for arbitrary nesting depth because it would need an unbounded count of outstanding opening parentheses. The PDA solves this by storing one stack marker for every opening parenthesis and removing the marker when its matching close is read.

- The stack records the number and order of currently unmatched opening parentheses.
- The algorithm detects an extra closing parenthesis immediately.
- An opening parenthesis left on the stack at the end indicates a missing closing parenthesis.
- The implementation is deterministic because the action for each input symbol and current stack condition is uniquely determined.

6. CASE STUDY 3 – PALINDROME CHECKER

6.1 Problem Definition

Palindrome recognition is a string-comparison problem in which the order of symbols is important. Examples over the selected alphabet include a, aa, aba, abba, and baab. Strings such as aab and abab are not palindromes because at least one pair of corresponding symbols from the two ends differs. The checker must compare symbols symmetrically, even though it reads the input from left to right.

The objective is to recognize palindrome strings over the alphabet {a, b}. A palindrome reads the same from left to right and right to left. The empty string is included in the language.

6.2 Target Language

$L = \{ w \in \{a,b\}^* \mid w = w^R \}$, where w^R denotes the reverse of w.

6.3 Context-Free Grammar

The grammar builds a palindrome from the outside toward the centre. Each recursive production places the same terminal at the beginning and end of the string, preserving the palindrome property. The derivation stops with ε for an even-length palindrome or with a single a or b for an odd-length palindrome. For example, abba can be derived by $S \Rightarrow aSa \Rightarrow abSba \Rightarrow abba$, using $S \Rightarrow \varepsilon$ at the centre.

$S \rightarrow aSa \mid bSb \mid a \mid b \mid \varepsilon$

The recursive productions add the same symbol to both ends of a shorter palindrome. The productions $S \rightarrow a$ and $S \rightarrow b$ generate one-symbol palindromes, while $S \rightarrow \varepsilon$ generates the empty palindrome.

6.4 NPDA Design

The input midpoint is not marked in advance, so the NPDA explores possible midpoint choices. Before the midpoint, symbols are pushed onto the stack. At a guessed midpoint, the machine changes to its matching phase; for odd-length strings it consumes one central symbol without placing it on the stack. After this change, every input symbol must equal the symbol at the top of the stack. A successful computation empties the stack exactly when the input is exhausted. Because several midpoint choices may be possible, the machine is nondeterministic; acceptance occurs if at least one computation path succeeds.

The NPDA has two main processing states: PUSH and MATCH. In PUSH, it stores a guessed first half of the input on the stack. It can nondeterministically guess an even-length midpoint and move to MATCH without consuming a symbol, or guess an odd-length midpoint by skipping one input symbol. In MATCH, each remaining input symbol must match the stack top; the matching symbol is popped. The input is accepted when all input is consumed and the stack is empty in MATCH.

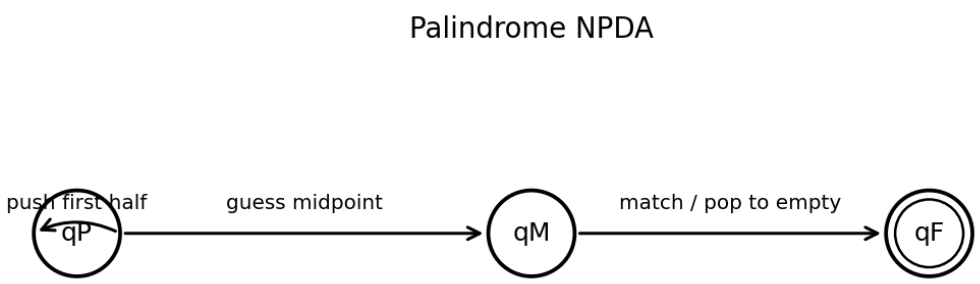


Figure 3. Simplified NPDA flow for palindrome recognition.

State	Purpose	Operation
PUSH	Store first half	Read a or b and push it onto the stack
PUSH → MATCH	Guess the middle	Choose an even midpoint or skip one middle symbol for odd length
MATCH	Compare second half	Compare current input symbol with stack top and pop on a match
Accept	Finish recognition	Input consumed and stack empty

6.5 Worked Processing Example

For the input abba, the NPDA chooses an even-length midpoint after reading ab. The stack contains [a, b]. It then reads the remaining b and a, matching and popping each symbol.

Step	State	Input position	Stack	Action
1	PUSH	0	[]	START
2	PUSH	1	[a]	PUSH 'a'
3	PUSH	2	[a, b]	PUSH 'b'
4	MATCH	2	[a, b]	GUESS EVEN MIDPOINT
5	MATCH	3	[a]	MATCH AND POP 'b'
6	MATCH	4	[]	MATCH AND POP 'a'

The input is fully consumed and the stack is empty, so abba is accepted.

6.6 Test Case Validation

Input	Expected	Actual	Status
a	Accepted	Accepted	PASS
aa	Accepted	Accepted	PASS
aba	Accepted	Accepted	PASS
abba	Accepted	Accepted	PASS
baab	Accepted	Accepted	PASS

ε (empty string)	Accepted	Accepted	PASS
aab	Rejected	Rejected	PASS
abb	Rejected	Rejected	PASS
abab	Rejected	Rejected	PASS
abc	Rejected	Rejected	PASS

Result: 10 test cases were executed. All 10 passed and no failures were reported.

6.7 Analysis

The NPDA approach directly models the comparison between the first half and the reverse of the second half. For an even-length input, the midpoint lies between two symbols. For an odd-length input, the central symbol is not required to match another symbol and is skipped. A finite automaton has no unbounded memory for storing the first half, so it cannot perform this comparison for palindromes of arbitrary length. The stack stores the required symbols and makes the context-free recognition possible.

- The stack stores the first half so that the second half can be checked in reverse order.
- Nondeterminism is used to guess the midpoint because the input length and midpoint are not known in advance.
- The odd-length case is handled by skipping exactly one middle symbol.
- Inputs containing symbols outside {a, b} are rejected by the implementation.

7. COMPARATIVE ANALYSIS

Feature	Arithmetic expressions	Balanced parentheses	Palindromes
Formal model	CFG + predictive PDA	CFG + DPDA-style stack procedure	CFG + NPDA
Main stack use	Grammar expansion and terminal matching	Track unmatched '(' symbols	Store first half and compare in reverse
Determinism	Predictive choices based on lookahead	Deterministic	Nondeterministic midpoint guess
Acceptance condition	All tokens and $ matched; stack empty	Input consumed; stack empty	Input consumed; stack empty in MATCH
Key challenge	Precedence and nested grouping	Correct nesting and matching counts	Comparing mirrored symbols

8. RESULTS AND DISCUSSION

Case study	Number of tests	Passed	Failed	Observed result
Arithmetic expression validation	8	8	0	All tested valid and invalid expressions classified correctly
Balanced parentheses checker	8	8	0	All tested balanced and unbalanced strings classified correctly
Palindrome checker	10	10	0	All tested palindromes and non-palindromes classified correctly
Overall	26	26	0	100% pass rate on the supplied test cases

The test outputs demonstrate that each program produced the expected result for every supplied test string. The detailed traces also illustrate the stack operations used to reach acceptance. These results validate the selected examples; they do not constitute a proof of correctness for every possible input.

9. CONCLUSION

The three case studies demonstrate the practical use of Context-Free Grammars and Pushdown Automata in syntax and pattern validation. The arithmetic-expression validator uses a grammar that encodes operator precedence and a predictive stack process. The balanced-parentheses checker uses deterministic push and pop operations to verify properly nested symbols. The palindrome checker uses a nondeterministic PDA to guess the midpoint and compare the second half against the stack.

All 26 test cases included in the submitted notebook passed: 8 for arithmetic expressions, 8 for balanced parentheses, and 10 for palindrome recognition. The study shows why stack memory is important for context-free patterns involving nesting or reverse matching, and how CFG/PDA models provide a formal foundation for such validation tasks.

10. REFERENCES

1. Course Assignment Instructions and Rubrics, Continuous Learning Assessment – 2, 21CSC301T – Formal Language and Automata, SRM Institute of Science and Technology, Task 3.
2. Submitted Python notebook and execution outputs for Task 3: Arithmetic Expression Validation, Balanced Parentheses Checker, and Palindrome Checker.
3. Hopcroft, J. E., Motwani, R., and Ullman, J. D., Introduction to Automata Theory, Languages, and Computation, Pearson.